数据库基础

数据库基础

- 不同角色的人对数据库知识学习的侧重点：
  - 研发人员(测试及开发人员)：主要学习如何操作数据库的逻辑对象，如表、索引和视图等，以得到应用程序中需要的数据。
  - 数据库管理员(DBA)：要学习和掌握数据库的全部内容，包括数据库对象管理、性能优化、备份与恢复、安全管理等

数据库基础

- 软件测试在什么时候会用到数据库的知识？
  - 搭建测试环境
  - 通用数据库操作功能的测试(增、删、改、查等)
  - 快速创建测试数据，提高测试工作效率

数据库基础

- 数据库的基本概念：
- 数据（Data）

  数据库中存储的基本对象
- 数据库（DataBase）

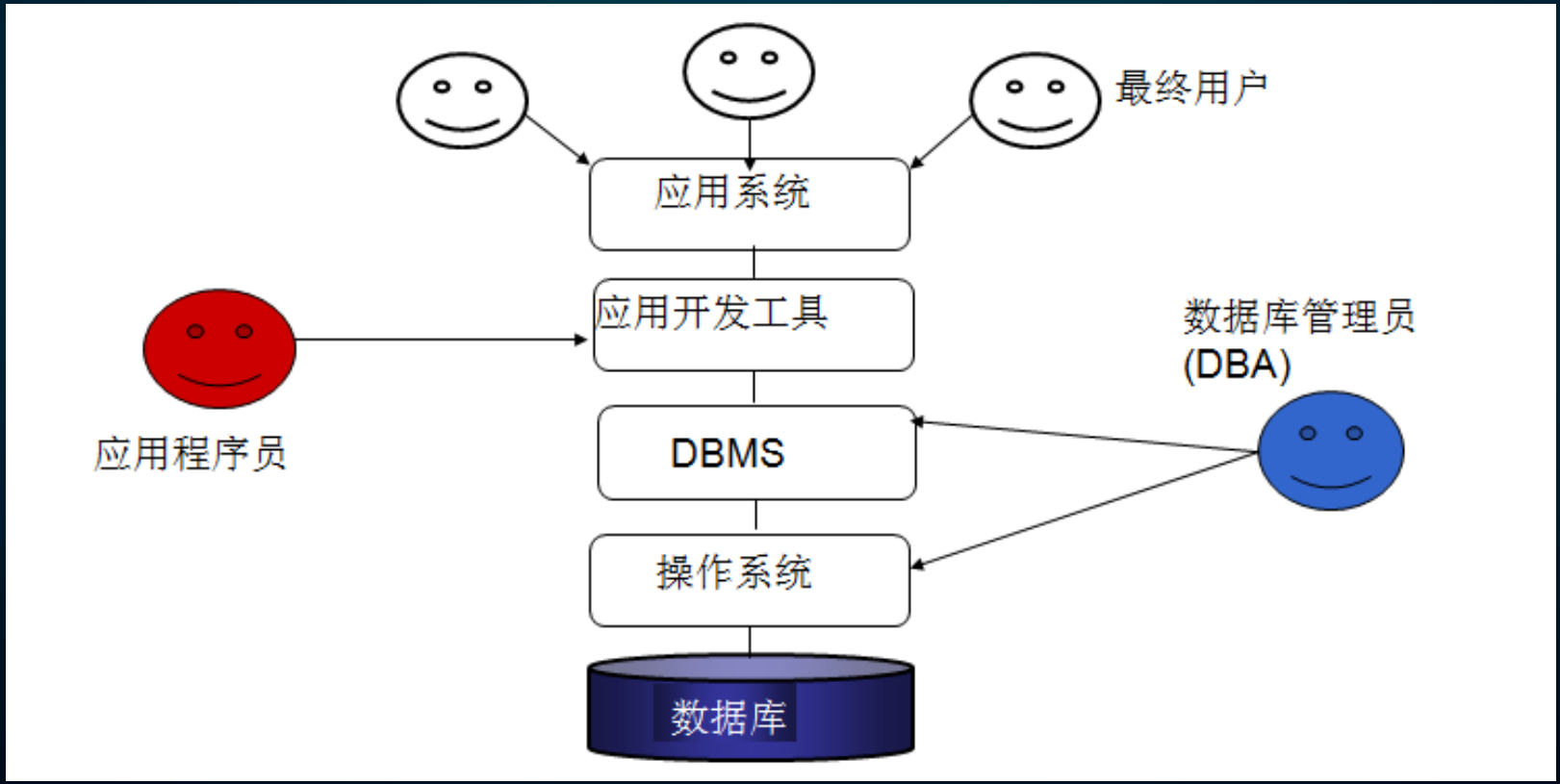
  是指长期储存在计算机内的、有组织的、可共享的大量数据集合。数据库中的数据按一定的数据模型组织、描述和储存，具有较小的冗余度、较高的数据独立性和易扩展性，并可为各种用户共享

数据库基础

- 数据库管理系统(DataBase Management System)
  一个对数据库进行统一管理、统一控制的系统软件，用户对数据库的任何操作都是通过它来完成的，因此也被称为数据平台，是用户和数据库之间的桥梁
- 数据库系统(DataBase System)
  计算机系统中引入数据库后的一个人机系统，一般由数据库、数据库管理系统、计算机硬件、操作系统、开发工具、应用系统、数据库管理员和用户构成

数据库基础

- 数据库系统



数据库基础

- 数据库系统的出现使信息系统的开发从以加工数据的应用程序为中心转向以共享的数据库为中心的新阶段，这和数据在各行各业的基础地位是相符合的
- 优势：

  数据库系统的使用便于数据的集中管理，又有利于应用程序的研发和维护，提高了数据的利用率，更重要的还提高了数据的安全性、正确性和可靠性

数据库管理系统

- 数据库管理系统的数据模型：

  由于计算机不能直接处理显示世界中的客观对象，所以必须先对其进行抽象，将其转换成计算机能够处理的数据，为此数据库使用数据模型来对现实世界进行抽象和模拟，并且按照数据模型对数据进行组织
- 数据模型的类型：
  - 层次模型
  - 网状模型
  - 关系模型（主流）

数据库管理系统

- 关系模型在1970年由美国IBM公司San Jose研究室的研究员E.F.Codd提出，是一种使用表格表示实体和实体之间关系的数据模型
- 关系数据模型中，无论是实体、还是实体之间的联系都被映射成一张二维表

学生登记表

学　号	姓　名	年　龄	性　别	系　名	年　级
95004	王小明	19	女	社会学	95
95006	黄大鹏	20	男	商品学	95
95008	张文斌	18	女	法律学	95
…	…	…	…	…	…

数据库管理系统

- 实体：客观存在并可相互区别的事物称为实体，可以是具体的人、事、物
- 属性：实体所具有的某一特性称为属性，一个实体可以由若干个属性来刻画
- 实体型：用实体名及其属性名集合来抽象和刻画同类实体称为实体型
- 实体集：同型实体的集合称为实体集
- 联系：实体内部和实体之间的联系（一对一、一对多、多对多）

数据库管理系统

- 关系数据库产品
  - Oracle：Oracle、MySQL
  - IBM：DB2、INFORMIX
  - 微软：SQLServer

345 systems in ranking, March 2019

Rank			DBMS	Database Model	Score		
Mar 2019	Feb 2019	Mar 2018			Mar 2019	Feb 2019	Mar 2018
1.	1.	1.	Oracle	Relational, Multi-model	1279.14	+15.12	-10.47
2.	2.	2.	MySQL	Relational, Multi-model	1198.25	+30.96	-30.62
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	1047.85	+7.79	-56.94
4.	4.	4.	PostgreSQL	Relational, Multi-model	469.81	-3.75	+70.46
5.	5.	5.	MongoDB	Document	401.34	+6.24	+60.82
6.	6.	6.	IBM Db2	Relational, Multi-model	177.20	-2.23	-9.47
7.	↑ 9.	7.	Microsoft Access	Relational	146.20	+2.18	+14.26
8.	↓ 7.	8.	Redis	Key-value, Multi-model	146.12	-3.32	+14.90
9.	↓ 8.	9.	Elasticsearch	Search engine, Multi-model	142.79	-2.46	+14.25
10.	10.	↑ 11.	SQLite	Relational	124.87	-1.29	+10.06

数据库管理系统

- 三大关系型数据库对比：

1. Oracle是一种大型数据库系统，能够运行在所有主流的平台上（Windows、UNIX、Linux、VMS等），功能强大，能够处理大批量的数据，性能和稳定性最好

   不过Oracle的使用成本比较昂贵，一套正版的Oracle软件早在2006年时价格就已经达到了6位数，因此一般应用于商业、政府部门等超大型项目中

数据库管理系统

2. SQLServer是微软推出的一款数据库产品，实现了真正的客户机/服务器体系结构，拥有图形化的用户界面，使系统管理和数据库管理更加直观、简单，同时SQLServer与Windows系统很好的进行了集成

   不过SQLServer的劣势也是非常明显的：只能运行于Windows操作系统，并且在实际使用中SQLServer在大数据量和大交易量的环境下表现不是很理想

数据库管理系统

3. MySQL是一个小型数据库管理系统（千万级的数据量），适用于所有的系统平台，最显著的特点是开源，因此价格便宜，使用成本低
   MySQL数据库不具备其他一些大型数据库的高级功能，但也正是因为这一点，使得MySQL体积小、运行速度快、安装和使用方便，目前被广泛的应用在中小型系统中，特别是在网络应用中用户群更多

SQL命令

SQL命令

- 结构化查询语言(Structured Query Language)简称SQL，是一种特殊目的的编程语言，是一种数据库查询和程序设计语言，用于存取数据以及查询、更新和管理关系数据库系统
- SQL命令的分类：
  - DDL 数据定义语句
  - DML 数据操纵语句
  - DQL 数据查询语句
  - DCL 数据控制语句

SQL命令-DDL

- DDL（数据定义语言）是用来定义数据结构，完成数据库对象的创建（CREATE）、修改（ALTER）、删除（DROP）

- 创建数据库

  语法：CREATE DATABASE 数据库名称
- 查看已有的数据库：SHOW DATABASES
- 使用已创建的数据库：USE 数据库名称

SQL命令-DDL

- 在使用CREATE语句创建数据库实例时，要求满足数据库命名规范：

1. 数据库名称不能重复
2. 数据库命名可以使用字母、数字和下划线，不能出现其他字符
3. 数据库命名不能超过30个字符
4. 数据库命名不能与系统关键字冲突
5. 数据库命名尽量有意义，比如体现数据库属于哪个系统使用

SQL命令-DDL

- 创建数据库时，有时为了避免乱码，需要指定字符集
- 语法：CREATE DATABASE 数据库名

  DEFAULT CHARACTER SET 字符集 COLLATE 校对集
- 例子：CREATE DATABASE test_db DEFAULT CHARACTER SET utf8 COLLATE utf8_general_ci;

SQL命令-DDL

- 查看MySQL支持的字符集命令：SHOW CHARACTER SET
- 查看MySQL字符集支持的校对集命令：SHOW COLLATION

信息 结果1 状态

Charset	Description	Default collation	Maxlen
big5	Big5 Traditional Chinese	big5_chinese_ci	2
dec8	DEC West European	dec8_swedish_ci	1
cp850	DOS West European	cp850_general_ci	1
hp8	HP West European	hp8_english_ci	1
koi8r	KOI8-R Relcom Russian	koi8r_general_ci	1

信息 结果1 状态

Collation	Charset	Id	Default	Compiled	Sortlen
latin5_bin	latin5	78			0
armscii8_general_ci	armscii8	32	Yes		0
armscii8_bin	armscii8	64			0
utf8_general_ci	utf8	33	Yes	Yes	1
utf8_bin	utf8	83		Yes	1
utf8_unicode_ci	utf8	192		Yes	8

SQL命令-DDL

- 创建数据表

  语法：CREATE TABLE 表名(属性名称 数据类型,……)

```sql
USE test;

CREATE TABLE test_table1
(
  t_id INT,
  t_name VARCHAR(10),
  t_age INT
);
```

- 查看已有的数据表命令：SHOW TABLES
- 查看已创建的数据表属性命令：DESC 表名

SQL命令-DDL

- MySQL数据库支持的数据类型：

1. 整型

   INT，在MySQL数据库支持存储最大10位整数

2. 浮点型

   FLOAT(m,d)，m表示数据总长度，d表示小数位数

3. 字符串

   CHAR(m),VARCHAR(m)，m表示字符串数据的总长度

SQL命令-DDL

4. 时间日期

   DATE，日期，格式：2019-01-01

   TIME，时间，格式：09:30:30

   DATETIME，日期和时间，格式：2019-01-01 09:30:30

   TIMESTAMP，自动存储当前系统日期和时间

SQL命令-DDL

5. enum数据类型、set数据类型

当数据表中的属性定义为这两种类型时，代表只能从指定的集合中取值

```
CREATE TABLE test_table1
(
  n1 ENUM('a','b','c'),
  n2 SET('a','b','c')
);

INSERT INTO test_table1 VALUES('a','a,b');
```

- 它们之间的区别是：enum类型只能取一个值，set类型可以取多个值
- enum数据类型定义的集合最多允许有65535个成员值
- set数据类型定义的集合最多允许有64个成员值

SQL命令-DDL

- 修改数据库

  使用alter命令来完成数据库的修改，一般是修改数据库的字符集和校对集

- 语法：ALTER DATABASE 数据库名称

  DEFAULT CHARACTER SET 字符集 COLLATE 校对集

SQL命令-DDL

- 修改数据表

1. 新增属性

   语法： ALTER TABLE 表名 ADD 属性名称 数据类型

   例子： ALTER TABLE test_table ADD name VARCHAR(20);

2. 修改属性

   语法：ALTER TABLE 表名 MODIFY 属性名称 数据类型

   语法：ALTER TABLE 表名 CHANGE 属性名称 属性名称 数据类型

   例子：ALTER TABLE test_table MODIFY name VARCHAR(20);

   ALTER TABLE test_table CHANGE name name2 VARCHAR(20);

SQL命令-DDL

3. 删除属性

   语法：ALTER TABLE 表名 DROP 属性名称

   例子：ALTER TABLE test_table DROP name;

- 如果表中只有一个属性，不允许删除

SQL命令-DDL

4. 修改表名

语法：ALTER TABLE 表名 RENAME TO 新表名

语法：RENAME TABLE 原表名 TO 新表名

例子：

```
ALTER TABLE test_table RENAME TO test_table_new;
RENAME TABLE test_table TO test_table_new;
```

SQL命令-DDL

- 删除数据库

  语法：DROP DATABASE 数据库名称

  例子： DROP DATABASE test_db;

- 删除数据表

  语法：DROP TABLE 表名

  例子： DROP TABLE test_table;

SQL命令-DDL

- 练习

1. 创建名称为hzdl_company的数据库
2. 在hzdl_company数据库中创建3张数据表

- hzdl_emp 员工信息表

字段名	数据类型	字段说明
empno	int	员工编号
ename	varchar(20)	员工姓名
jobno	int	职位编号
mgr	int	经理编号
hiredate	date	入职日期
deptno	int	部门编号

SQL命令-DDL

- hzdl_dept 部门信息表

字段名	数据类型	字段说明
deptno	int	部门编号
dname	varchar(20)	部门名称
loc	varchar(20)	部门所在地

- hzdl_job 职位信息表

字段名	数据类型	字段说明
jobno	int	职位编号
jobname	varchar(20)	职位名称
sal	float(7,2)	工资

SQL命令-DML

- DML（数据操纵语言）是用于维护数据表中的具体数据，完成数据的增加（INSERT）、更新（UPDATE）、删除（DELETE）

- 数据插入

  语法：INSERT INTO 表名(属性1,属性2,……) VALUES(值1,值2,……)

  例子：INSERT INTO test_table(id,name) VALUES(1,'zhangsan');

- 向数据表中插入数据时，值与属性要一一对应，且数据类型要匹配

SQL命令-DML

- 数据更新

  语法：UPDATE 表名 SET 属性1=新值1,属性2=新值2,…[WHERE条件子句]

  例子：UPDATE test_table SET name='lisi' WHERE id=1;
- UPDATE语句会将数据表中满足WHERE条件的对应元组的属性1、属性2…更新为新的值
- 如果后面不接WHERE条件子句，则会更新整张表的所有数据

SQL命令-DML

- 批量替换update语句
- UPDATE test_table SET name=REPLACE(name,"杰","王");

SQL命令-DML

- 数据删除

  语法：DELETE FROM 表名 [WHERE条件子句]

  例子：DELETE FROM test_table WHERE id=1;
- DELETE语句会将数据表中满足WHERE条件的对应元组删除
- 如果后面不接WHERE条件子句，则会删除数据表中的所有数据

SQL命令-DQL

- DQL（数据查询语言）主要用于进行数据表中数据的查询

  基本语法：SELECT 属性1,属性2,…. FROM 表名

  例子：SELECT empno,ename,jobno,hiredate FROM hzdl_emp;

- 通配符*，代表所有，可以使用*代表查询表中所有列

  例子：SELECT * FROM hzdl_emp;

SQL命令-DQL

- 使用SELECT语句进行数据查询时，还可以进行算术运算、限定查询、排序、分组等控制
- 算术运算

  能够在SELECT语句查询的结果基础上进行相应的数学运算

  例子：SELECT sal+100 FROM hzdl_job;

SQL命令-DQL

- 限定查询

  在使用SELECT语句进行查询时，使用WHERE子句，就能查询出满足条件的记录，而不是表中的所有记录

  例子：SELECT * FROM hzdl_emp WHERE empno=20161001;

- WHERE子句也可以使用多个限定条件

SQL命令-DQL

- 在WHERE子句中设置的限定条件，一般由各种运算符组成
- MySQL中常用的运算符有几类：数学运算符、比较运算符、逻辑运算符
- 比较运算符：

运算符	含义
=	等于
>	大于
>=	大于等于
<	小于
<=	小于等于
<>、!=	不等于

SQL命令-DQL

- 特殊的比较运算符：

运算符	含义
BETWEEN... AND...	在两值之间 (包含)
IN(list)	匹配列出的值
LIKE	匹配一个字符模式
IS NULL	匹配空值

```
SELECT * FROM hzdl_emp WHERE empno BETWEEN 20161001 AND 20162001;
SELECT * FROM hzdl_emp WHERE empno IN(20161001,20162001);
SELECT * FROM hzdl_emp WHERE mgr IS NULL;
SELECT * FROM hzdl_emp WHERE ename LIKE '财务%';
SELECT * FROM hzdl_emp WHERE ename LIKE '%经理';
SELECT * FROM hzdl_emp WHERE ename LIKE '_经理';
```

- 通配符%代表若干个字符，通配符_代表一个字符

SQL命令-DQL

- 逻辑运算符

运算符	含义
AND 或者 &&	如果组合的条件都为真则返回真值
OR 或者 ||	如果组合的条件之一是真值,返回真值
NOT 或者 !	如果条件为假则返回真值

```
SELECT * FROM hzdl_emp WHERE ename='董事长' AND empno=20161001;
SELECT * FROM hzdl_emp WHERE ename<>'董事长' && deptno=4;
SELECT * FROM hzdl_emp WHERE deptno=1 OR deptno=2;
SELECT * FROM hzdl_emp WHERE ename='董事长' || ename='总经理';
SELECT * FROM hzdl_emp WHERE NOT empno=20161001;
SELECT * FROM hzdl_emp WHERE !(empno=20161001);
```

- 逻辑运算符!在使用的时候一定要注意不能省略后面括号

SQL命令-DQL

- 运算符优先级

  数学运算符>比较运算符>逻辑运算符

SQL命令-DQL

- 去除重复

  使用DISTINCT关键字能够将查询到的结果集中重复的记录去除

  例子：SELECT DISTINCT hiredate FROM hzdl_emp;

SQL命令-DQL

- 设置别名

  使用AS关键字可以在查询时设置属性（列）的别名，不过AS可以省略

  例子：SELECT empno AS ’员工号’,ename AS ’员工姓名’ FROM hzdl_emp;

SQL命令-DQL

- 排序

  使用ORDER BY子句可以将SELECT查询到的结果集按照指定属性排序

  默认采用升序（ASC）排序方式，也可以使用降序（DESC）

  例子：SELECT * FROM hzdl_emp ORDER BY deptno;
- 同样，排序时也可以指定多个属性排序
- 当多属性排序方式有冲突时，按照指定先后顺序排序

SQL命令-DQL

- 分组统计

  使用GROUP BY子句可以对查询结果按指定属性进行分组统计

  例如：查询公司内各个部门员工人数

  SELECT COUNT(deptno) FROM hzdl_emp GROUP BY deptno;

- 一般分组GROUP BY子句和聚合函数一起使用，常用的聚合函数有：

  COUNT()、AVG()、SUM()、MAX()、MIN()、GROUP_CONCAT()

SQL命令-DQL

- 计数函数COUNT

  统计表中某个属性对应的记录数

  例子：SELECT COUNT(empno) FROM hzdl_emp;

- 求平均值函数AVG

  统计表中某个属性对应的平均值

  例子：SELECT AVG(sal) FROM hzdl_job;

SQL命令-DQL

- 求和函数SUM

  统计表中某个属性的汇总值（和）

  例子：SELECT SUM(sal) FROM hzdl_job;

- 最大值函数MAX

  统计表中某个属性的最大值

  例子：SELECT MAX(sal) FROM hzdl_job;

SQL命令-DQL

- 最小值函数MIN

  统计表中某个属性的最小值

  例子：SELECT MIN(sal) FROM hzdl_job;

- 列值拼接函数GROUP_CONCAT

  将表中某个属性的所有值拼接到同一个结果中，默认值与值之间用逗号分隔，也可以使用SEPARATOR关键字指定分隔符

  例子：SELECT GROUP_CONCAT(jobname SEPARATOR ’+’) FROM hzdl_job;

SQL命令-DQL

- 需要注意的是：

1. 在SELECT命令中使用分组时，SELECT关键字后只能使用聚合函数和GROUP BY子句后出现的属性
2. 聚合函数不能直接在WHERE子句中使用

SQL命令-DQL

- 和分组GROUP BY子句经常一起使用的还有HAVING子句

  HAVING子句为分组统计进一步设置统计条件，效果和WHERE子句类似

- 它们的区别是：

1. WHERE子句在查询前限定查询条件
2. HAVING子句在分组后进行进一步限定

例如：查询公司内各个部门员工人数，并返回部门员工人数大于2人的部门编号

```
SELECT COUNT(*),deptno  FROM hzdl_emp GROUP BY deptno HAVING
    COUNT(deptno)>2;
```

SQL命令-DQL

- 分组同样可以基于多个属性

  例如：查询公司内各个部门不同职位的员工人数

- SQL语句：

  SELECT deptno,jobno,COUNT(empno) FROM hzdl_emp GROUP BY deptno,jobno;

SQL命令-DQL

- 限定结果

  使用LIMIT子句可以限定查询返回的记录数

  例如：查询数据表中前两条记录

  SQL语句：SELECT * FROM hzdl_emp LIMIT 2;

SQL命令-DQL

- LIMIT子句后面可以有一个参数，也可以有两个参数

1. 如果有一个参数，该参数表示从表开始位置查询对应的记录数量
2. 如果有两个参数，那么第一个参数表示起始位置与第一行的偏移量，第二个参数表示查询的数量

例子：SELECT * FROM hzdl_emp LIMIT 2,3;

SQL命令-DQL

- 练习

1. 查询hzdl_emp表中所有员工的员工姓名
2. 查询hzdl_job表中年薪大于4W的职位
3. 查询hzdl_emp表中姓张的员工信息
4. 查询hzdl_emp表中2016年6月前入职的员工姓名
5. 查询hzdl_emp表中部门编号为5的员工人数
6. 查询hzdl_emp表中同一天入职的员工人数
7. 查询hzdl_emp表中最近刚入职的员工姓名　2019/11/11

SQL命令-DQL

- 单行函数

  数据库中的函数作用也非常大，一般多用于SELECT语句和WHERE语句

- 函数按照类型又可分为：单行函数和多行函数

➢ 单行函数：就是将数据表中的每条数据进行独立的计算并返回结果

➢ 多行函数：就是计算数据表中的多条数据并返回结果，前面接触的聚合函数就是多行函数，一般用于数据的统计汇总

SQL命令-DQL

字符串函数

- length(x)：返回字符串x的长度，以字节为单位
- char_length(x)：返回字符串x的长度，以字符为单位
- substring(x,start,length)：从start开始截取，长度为length的字符串x
- left(x,length)：从字符串x的左边返回长度为length的子字符串
- right(x,length)：从字符串x的右边返回长度为length的子字符串
- replace(x,char1,char2)：将字符串x中的char1替换为char2
- concat(x,y)：将字符串x和y拼接起来
- instr(x,char)：返回char在字符串x中第一次出现的位置

SQL命令-DQL

数值函数

- floor(x)：返回小于等于x的最大整数
- ceiling(x)：返回大于等于x的最小整数
- round(x)：返回x的整数部分（四舍五入）
- round(x,y)：返回x精确到y位的结果（四舍五入）
- truncate(x,y)：返回x精确到y位的结果（非四舍五入）
- abs(x)：返回x的绝对值

SQL命令-DQL

时间日期函数

- curdate()：返回当前系统的日期
- curtime()：返回当前系统的时间
- now()：返回当前系统的日期和时间
- date_add(x,interval n f)：返回时间x加上间隔时间n的结果，f用于指定时间单位 second/minute/hour/day/week/month/year
- datediff(date1,date2)：返回两个日期的差值
- timediff(time1,time2)：返回两个时间的差值
- date_format(x,fmt)：返回时间x按照指定格式fmt格式化的结果

SQL命令-DQL

- 时间格式化使用的格式符

```
%S,%s 两位数字形式的秒（ 00,01, ..., 59）
%I,%i 两位数字形式的分（ 00,01, ..., 59）
%H 两位数字形式的小时，24 小时（00,01, ..., 23）
%h 两位数字形式的小时，12 小时（01,02, ..., 12）
%k 数字形式的小时，24 小时（0,1, ..., 23）
%l 数字形式的小时，12 小时（1, 2, ..., 12）
%T 24 小时的时间形式（hh:mm:ss）
%r 12 小时的时间形式（hh:mm:ss AM 或hh:mm:ss PM）
%p AM或PM
%W 一周中每一天的名称（Sunday, Monday, ..., Saturday）
%a 一周中每一天名称的缩写（Sun, Mon, ..., Sat）
%d 两位数字表示月中的天数（00, 01,..., 31）
```

```
%e 数字形式表示月中的天数（1, 2,  ..., 31）
%D 英文后缀表示月中的天数（1st, 2nd, 3rd,...）
%w 以数字形式表示周中的天数（ 0 = Sunday, 1=Monday, ..., 6=Saturday）
%j 以三位数字表示年中的天数（ 001, 002, ..., 366）
%U 周（0, 1, 52），其中Sunday 为周中的第一天
%u 周（0, 1, 52），其中Monday 为周中的第一天
%M 月名（January, February, ..., December）
%b 缩写的月名（ January, February,...., December）
%m 两位数字表示的月份（01, 02, ..., 12）
%c 数字表示的月份（1, 2, ...., 12）
%Y 四位数字表示的年份
%y 两位数字表示的年份
%% 直接值“%”
```

SQL命令-DQL

数据类型转换函数

- convert(value,type)：将数据value转换为type类型
- 类型type可以使用：
  - char：字符串
  - decimal：浮点型
  - signed：整型
  - date：日期
  - time：时间

```
SELECT CONVERT(100,CHAR);
SELECT CONVERT('1.23',DECIMAL)+1;
SELECT CONVERT('100',SIGNED)+1;
SELECT CONVERT(NOW(),DATE);
SELECT CONVERT(NOW(),TIME);
```

SQL命令-DQL

- 练习

1. 查询hzdl_emp表中名字长度超过2个字的员工姓名
2. 查询hzdl_emp表中员工‘李四’入职前一天的日期

SQL命令-DQL

- 多表查询

  使用一个SELECT语句查询多张表，即多表查询
- 多表查询分为两类：
  - 内连接
  - 外连接

SQL命令-DQL

- 内连接

  进行多表查询时，按照匹配条件，在结果集中只保留各个表中满足匹配条件的记录

- 例子：

```
SELECT hzdl_emp.ename,hzdl_job.jobname FROM hzdl_emp,hzdl_job
WHERE hzdl_emp.jobno=hzdl_job.jobno;
SELECT hzdl_emp.ename,hzdl_job.jobname FROM hzdl_emp INNER JOIN
hzdl_job ON hzdl_emp.jobno=hzdl_job.jobno;
```

SQL命令-DQL

- 自然连接

  连接时不需要额外指定连接条件，因为会自动按照相同属性进行等值连接，且会删除结果集中的重复属性

- 例子：

```
SELECT * FROM hzdl_emp NATURAL JOIN hzdl_dept;
SELECT * FROM hzdl_emp INNER JOIN hzdl_dept USING(deptno);
```

SQL命令-DQL

- 自连接

  就是表与它自己进行的连接查询，即自连接

  在自连接查询过程中，一定要设置表的别名

- 例子：查询员工“张三”的上级主管姓名

  SELECT e2.ename AS ‘上级主管’ FROM hzdl_emp e1,hzdl_emp e2

  WHERE e1.mgr=e2.empno AND e1.ename=‘张三’;

SQL命令-DQL

- 外连接

  进行多表查询时，分为主、从表，在结果集中保留主表中的所有记录以及从表中满足匹配条件的记录

- 外连接又分为：
  - 左外连接
  - 右外连接

SQL命令-DQL

- 左外连接

  在连接查询过程中，以左表为主表，在结果集中保留左表中的全部记录及右表中满足匹配条件的记录

- 例子：

```
SELECT hzdl_emp.ename,hzdl_job.jobname FROM hzdl_emp LEFT OUTER JOIN hzdl_job ON hzdl_emp.jobno=hzdl_job.jobno;
```

SQL命令-DQL

- 右外连接

  在连接查询过程中，以右表为主表，在结果集中保留右表中的全部记录及左表中满足匹配条件的记录

- 例子：

```
SELECT hzdl_emp.ename,hzdl_job.jobname FROM hzdl_emp RIGHT OUTER JOIN hzdl_job ON hzdl_emp.jobno=hzdl_job.jobno;
```

SQL命令-DQL

- 子查询

  将一个SELECT语句A使用于另外一个SELECT语句B中，则A即为B中的子查询

- 使用子查询注意事项：

1. 子查询一定要使用括号括起来
2. 子查询放在比较运算符的右边
3. 对单行子查询使用单行运算符
4. 对多行子查询使用多行运算符

SQL命令-DQL

- 单行子查询

  即子查询语句只返回一行记录

- 单行比较运算符：

运算符	含义
=	等于
>	大于
>=	大于等于
<	小于
<=	小于等于
<>	不等于

张三的下属是哪几位？

- 例子：SELECT ename FROM hzdl_emp WHERE mgr = (SELECT empno FROM hzdl_emp WHERE ename=‘张三’);

SQL命令-DQL

- 多行子查询

  即子查询语句返回多行记录

- 多行比较运算符：

运算符	含义
IN	等于列表中的任何值
ANY	比较子查询返回的每一个值，只要其中一个值满足条件就返回true
ALL	比较子查询返回的每一个值，要其中每一个值都要满足条件才返回true

SQL命令-DQL

- 例子：

```
SELECT ename FROM hzdl_emp WHERE deptno IN (SELECT deptno FROM hzdl_dept WHERE loc= ‘绵阳’);
SELECT ename FROM hzdl_emp WHERE deptno > ANY(SELECT deptno FROM hzdl_dept WHERE loc=’成都’);
SELECT ename FROM hzdl_emp WHERE deptno > ALL(SELECT deptno FROM hzdl_dept WHERE loc=’成都’);
```

SQL命令-DQL

- 练习

分别使用连接查询、子查询等不同方式实现下面的查询

1. 查询员工“张三”的月工资
2. 查询工作地点为绵阳的员工姓名
3. 查询财务部的平均工资
4. 查询比“张三”入职早的员工姓名

SQL命令-DCL

- DCL（数据控制语言）主要是数据库管理员用来管理其他用户权限时使用的语句
- 可以授予的权限包括以下几组：
  - 列权限：和表中的一个具体列相关
  - 表权限：和一个具体数据表中的所有数据相关
  - 数据库权限：和一个具体数据库中的所有数据表相关
  - 全局权限：和MySQL的所有数据库相关
- 低级权限会被高级权限覆盖

SQL命令-DCL

- 数据库管理员使用GRANT语句进行授权

  语法：GRANT 权限 ON 数据库对象 TO 用户名@用户地址 IDENTIFIED BY 用户口令
- 通过GRANT语句可以授予的权限包括：SELECT、INSERT、UPDATE、DELETE、CREATE、DROP、ALTER等，可以使用ALL代表所有
- 注意语法中的用户名、用户地址、用户口令需要用引号括起来，并且口令不能为空

SQL命令-DCL

- 列权限

  GRANT SELECT(id) ON test.test_table1 TO 'a'@'localhost' IDENTIFIED BY '123456';

- 表权限

  GRANT SELECT ON test.test_table1 TO 'b'@'localhost' IDENTIFIED BY '123456';

- 数据库权限

  GRANT SELECT ON test.* TO 'c'@'localhost' IDENTIFIED BY '123456';

- 全局权限

  GRANT SELECT ON *.* TO 'd'@'localhost' IDENTIFIED BY '123456';

SQL命令-DCL

- 查看指定用户权限：

  SHOW GRANTS FOR 用户名@用户地址

- 也可以使用USE命令切换到系统数据库information_schema

  SELECT * FROM user_privileges查看更为详细的用户权限

SQL命令-DCL

- 数据库管理员使用REVOKE语句取消授权

  语法：REVOKE 权限 ON 数据库对象 FROM 用户名@用户地址

- 例子：

  REVOKE ALL ON *.* FROM 'user'@'localhost';

SQL命令-DCL

- 在数据库管理员授予或取消授予权限时，用户地址可以使用%表示所有
- 在数据库管理员授予或取消授予权限后，需要刷新系统权限表：

  FLUSH PRIVILEGES
- 或者重新启动数据库服务：

  停止服务命令 net stop mysql

  启动服务命令 net start mysql

数据库对象

- 常见的数据库对象除了前面介绍的数据库和数据表，还包括：
  - 索引
  - 视图
  - 存储过程

数据库对象

- 索引

  数据库中使用索引，目的在于提高查询效率，本质上就是针对设置了索引的属性列按照特定顺序排序，便于查询

- 以一本书为例，索引就像书的目录一样，要找到第8章：

  如果没有目录，需要从头开始，一页一页翻到第8章

  如果有了目录，就可以按照目录的标识，翻到对应的页码

数据库对象

- 创建索引的语法：CREATE INDEX 索引名称 ON 表名(属性名称)

  例子：CREATE INDEX index_a ON test_a(name);
- 同一个数据表上可以定义多个索引，但实际上在查询时，也只能使用一个索引：

1. 如果数据表中有主键，则按照主键上的索引查询
2. 如果没有主键，会选择限制最为严格的索引

数据库对象

- 要查看某个数据表上创建的索引，可以使用SHOW命令

  语法：SHOW INDEX FROM 表名

  例子：SHOW INDEX FROM test;

- 要删除索引，可以使用DROP命令

  语法：DROP INDEX 索引名称 ON 表名

  例子：DROP INDEX index_test ON test;

数据库对象

- 索引的优势是提高查询效率
- 但同样也有缺点：

1. 索引的使用会降低更新表的速度，如果要对数据表进行INSERT、UPDATE、DELETE等更新操作，数据库不仅仅要保存更新之后的数据，也需要更新对应的索引，因此会导致更新效率降低
2. 索引的使用会创建对应的索引文件，占用磁盘空间

数据库对象

- 视图

  数据库中的视图是一个虚拟的表，在数据库中并不实际存在

  视图是由查询数据库中的数据表产生的，因此视图的内容由查询定义，实际上也可以把视图理解为保存查询结果的虚拟表

- 虽然视图是一个虚拟的表，但是它和像真实的数据表一样使用

数据库对象

- 使用视图的原因主要有：

1. 安全性更强，通过视图的使用可以将一些数据隐藏，仅将视图中的数据提供给用户使用
2. 提高查询性能，将经常访问的数据提前保存在视图中，后续的数据查询可以直接在视图上进行
3. 使复杂的查询易于理解和使用，将问题进行分解，利用多个视图获取数据并最终联合起来就能得到需要的结果

数据库对象

- 可以使用CREATE语句来创建视图

  语法：CREATE VIEW 视图名称 AS SELECT语句

- 例子：

```
CREATE VIEW view_test
AS
SELECT * FROM table1 WHERE id=1;
```

当给了限定条件时，就创建限定条件的内容

数据库对象

- 创建视图时，需要遵循的规则：

  视图也是一个表（虚拟表），因此不能与已有的表或视图名称重复

- 如果要查看数据库上的视图，可以使用命令：

  SHOW TABLE STATUS WHERE COMMENT='VIEW';

数据库对象

- 可以使用ALTER语句更新视图

  语法： ALTER VIEW 视图名称 AS SELECT语句

  例子：ALTER VIEW view_test AS SELECT * FROM table1 WHERE id=2;

- 可以使用DROP语句删除视图

  语法：DROP VIEW 视图名称

  例子：DROP VIEW view_test;

数据库对象

- 在实际应用中，视图的另外一个主要用途：通过视图来更新真实表

  因为视图本身是虚拟表，实际上是对原表的引用，因此在视图中做的数据更新操作会直接反馈到原表中

  例子：UPDATE view_test SET name= ‘zhangsan’;
- 同样因为这一特性，在原表中做的数据更新，也会反馈到视图中

数据库对象

- 需要注意的是，通过视图来更新真实表这一特性是有限制的：
  - 如果视图中的属性来自于两个以上的数据表，则此视图不允许更新
  - 如果视图中的属性来自于函数结果，则此视图不允许更新
  - 如果视图定义中含有GROUP BY子句，则此视图不允许更新
  - 如果视图定义中含有DISTINCT语句，则此视图不允许更新
  - 一个不允许更新的视图上定义的视图也不允许更新

数据库对象

- 存储过程

  存储过程是一组为了完成特定功能的SQL语句集，经过编译后存储在数据库中，用户可以通过存储过程的名称来调用并执行它
- 需要注意的是MySQL在5.0以前并不支持存储过程

数据库对象

- 存储过程的优点：

1. 增强了SQL语言的功能和灵活性，可以用控制流语句编写，完成负责的判断和运算
2. 实现较快的执行速度，因为存储过程是经过预编译后存储在数据库中的，调用并执行存储过程不需要再次编译
3. 减少网络传输量，因为在调用执行存储过程时，只需要将存储过程的名称和参数进行传送
4. 增强了数据的安全性，用户需要知道的仅仅是存储过程的名称和参数，避免了对数据的直接操作

数据库对象

- 使用CREATE语句创建存储过程

  语法：CREATE PROCEDURE 存储过程名称(参数列表)

  程序体

- 存储过程的参数列表中每一个参数由3部分构成：输入输出类型、参数名称、参数类型

  输入输出类型有3种：IN、OUT、INOUT

- 存储过程的程序体以BEGIN开始，END结束，可以包含一组语句集

数据库对象

- 例子：创建一个存储过程，能够根据输入的部门编号，统计出部门员工人数

```
CREATE PROCEDURE pro(IN dno INT,OUT count_num INT)
BEGIN
SELECT COUNT(empno) INTO count_num FROM hzdl_emp WHERE
deptno=dno;
END;
```

数据库对象

- 存储过程创建完成后，使用CALL来调用存储过程

  语法：CALL 存储过程名称(参数列表);

- 存储过程调用时，需要传递具体数据给输入参数，使用变量接收输出参数

  例子：统计编号为1的部门员工人数

  ```
  CALL pro(1,@cn);
  SELECT @cn;
  ```

数据库对象

- 查看数据库中所有存储过程基本信息

  语法：SHOW PROCEDURE STATUS;

- 存储过程的程序体也不能被修改，当存储过程不再被使用时，使用DROP语句删除

  语法：DROP PROCEDURE 存储过程名称;

控制流语句

- 变量定义

  在MySQL数据库的函数和存储过程中可以使用DECLARE关键字来定义变量，这些变量的作用范围是BEGIN…END程序体中
- 语法：DECLARE 变量名 变量类型 DEFAULT 默认值
- 例子：DECLARE abc INT DEFAULT 10;

控制流语句

- 变量赋值

  给BEGIN…END程序体中的变量赋值有两种方法

  语法：SET 变量名=值;

  语法：SELECT 属性 INTO 变量名 FROM 表名;

- 例子：

  SET @abc=10;

  SELECT id INTO @abc FROM table;

控制流语句

- IF语句

  IF语句主要用于条件判定，根据条件成立与否，后续执行不同的命令

- 语法：IF 条件 THEN …

  ELSEIF 条件 THEN …

  ELSE …

  END IF

控制流语句

- 例子：

```
CREATE PROCEDURE pro(IN x INT)
BEGIN
IF x=1 THEN SELECT 'x is 1';
ELSEIF x=2 THEN SELECT 'x is 2';
ELSE SELECT 'WRONG';
END IF;
END;
```

控制流语句

- WHILE语句

  WHILE循环语句，判断条件成立与否，如果成立则循环执行程序体

- 语法：w:WHILE 条件 DO

  …

  END WHILE

控制流语句

- 例子：

```
CREATE PROCEDURE pro(IN x INT)
BEGIN
WHILE x<5 DO
SELECT ’OK’;
SET x=x+1;
END WHILE;
END;
```

控制流语句

- 练习

1. 创建一个存储过程，通过接收员工工号，能够计算出员工每个月工资
2. 创建一个存储过程，通过接收员工工号，能够计算出员工的年薪（总经理为14薪，部门经理为13薪，职员为12薪）
3. 创建一个存储过程，向表hzdl_job表中插入100条数据，要求职位编号为101至200，职位名称为‘职位101’至‘职位200’，工资都设置为1000